14.2 Script de importação — status de entrega

Bloco 14.2 Script de importação · Sprint 14 — Migração da planilha e go-live · Data: 2026-10-14 · Branch: `main` · Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`)

Resumo

Fechado em 2026-10-27. Entregue: o importador em `scripts/importacao/` (simulação, idempotência, log de ignoradas, `--desfazer`), pronto e validado sobre a planilha falsa (ensaio 17/17, escrita 529/529, rodada de 27/10 nas sete camadas).

Fora do alcance (decisão do Breno, 27/10): a importação real, que depende da planilha "Gestão Contratos.xlsx". Se a planilha um dia aparecer: ajustar `scripts/importacao/mapa-planilha.json` aos cabeçalhos reais e rodar a simulação; depois `--gravar` em gc-dev e `conciliar.mjs` contra os totais do card, repetindo até bater.

Histórico: a rodada de 2026-10-19 passa nas sete camadas, com o ensaio da importação (9/9, então) dentro da escrita. A planilha real não virá (seção 6). As decisões do 14.1 foram tomadas por análise e **aplicadas no código em 26/10**: o ensaio agora tem 17 passos (17/17).

Implementado sem a planilha real, de propósito. A "Gestão Contratos.xlsx" ainda não chegou, e as decisões do 14.1 também não. O que não depende delas ficou pronto e testado:

- a leitura das abas;
- a montagem dos registros com cada regra do card;
- a gravação na ordem das FKs, sem duplicar;
- a simulação;
- o log das linhas ignoradas;
- o `desfazer`, para repetir em dev.

O que depende delas está isolado em dois lugares: o **mapa** (`mapa-planilha.json`), com os nomes de aba e de coluna, e as **funções de regra** (`src/lib/importacao.ts`), uma por decisão. Quando a planilha chegar, o trabalho é ajustar o mapa e conferir a simulação.

Para provar o mecanismo, existe uma **planilha falsa** no formato que se espera, com os casos difíceis de propósito. O ensaio sobre ela entrou na camada escrita do plano de validação.

1. As atividades do card

ATIVIDADE	ESTADO
Script que lê a planilha e grava via API do Supabase	✅ `scripts/importacao/importar-planilha.mjs` (exceljs + chave de serviço). Simulação por padrão; `--gravar` para gravar

ATIVIDADE	ESTADO
Ordem respeitando as FKs	✅ clientes → obras → propostas → itens → execução → NF → pagamentos → FD (`gravar.mjs`). A "empresa" é a do autor (`--autor` , padrão o usuário da validação)
Regra de cascata da decisão 3	✅ `cascataDaExecucao` : fab = ent = o maior entre a fabricada e a instalada, med = 0 salvo se vier; linha que não cabe (instalada acima do total, medida acima da instalada) é ignorada com o motivo, e o valor deduzido vira aviso. **Aguarda a produção validar**
As linhas de execução viram N execuções por item (sequencial)	✅ cada linha da aba é uma execução, sequencial pela ordem. Item com uma linha só usa a quantidade do item; com várias, cada linha precisa dizer a sua
Status de NF: "Paga" → paga, "Atrasada" → emitida	✅ com um ajuste: o status **não é gravado**. A NF nasce emitida (ou cancelada), e o trigger dos pagamentos a recalcula. O texto da planilha fica guardado para a conciliação (14.3) conferir que bate. Gravar "paga" à força criaria NF paga sem pagamento
Fornecedores de FD como texto	✅ campo `fornecedor`
Log de cada linha ignorada com o motivo	✅ na tela e no relatório JSON (`--relatorio`), com a referência `Aba!linha`
Idempotência	✅ chave de negócio por entidade: nome do cliente (sem acento e sem caixa), código da obra, número da proposta, proposta + item, item + sequencial, número + série da NF. Pagamento e FD não têm chave no banco: levam `[importacao: <hash>]` na observação. Rodar de novo: 0 criados
Modo dry-run	✅ é o padrão; só `--gravar` grava

Além do card: `--ate AAAA-MM-DD` (data de corte, decisão 5), `--desfazer` (apaga o que a planilha importou, para repetir em dev) e os links do Drive na observação (decisão 4, opção "manter como link").

2. Como usar

```
# simulação: lê, monta e relata; não grava
node --env-file=.env.local scripts/importacao/importar-planilha.mjs "Gestão Contratos.xlsx"

# grava em gc-dev (rodar de novo não duplica)
node --env-file=.env.local scripts/importacao/importar-planilha.mjs "Gestão Contratos.xlsx" --gravar

# desfaz o que esta planilha importou
node --env-file=.env.local scripts/importacao/importar-planilha.mjs "Gestão Contratos.xlsx" --desfazer
```

A saída mostra, por aba, onde achou o cabeçalho e quantas linhas leu; quantos registros vão ser gravados; os totais (NF emitida, recebido, FD a descontar); e as linhas ignoradas com o motivo.

3. Arquivos

ARQUIVO	O QUÊ
`src/lib/importacao.ts`	regras puras: números e datas no formato brasileiro, status de NF e de proposta, forma de pagamento, unidades, cascata (decisão 3), cliente canônico (decisão 1), chave de idempotência. 9 testes em `importacao.test.ts`
`scripts/importacao/mapa-planilha.json`	**provisório**: abas e colunas esperadas (cada campo aceita uma lista de cabeçalhos), equivalências de cliente, data de corte e tipos de NF
`scripts/importacao/ler-planilha.mjs`	acha a aba pelo nome e a linha de cabeçalho sozinha (a que casa mais colunas nas 15 primeiras)
`scripts/importacao/montar-registros.mjs`	linhas → registros, com os vínculos entre abas, as ignoradas e os avisos
`scripts/importacao/gravar.mjs`	gravação idempotente e o `desfazer`
`scripts/importacao/importar-planilha.mjs`	a linha de comando e o relatório
`scripts/importacao/planilha-falsa.mjs`	a planilha falsa com os casos difíceis
`scripts/validacao/validar-importacao.mjs`	o ensaio, na camada escrita

4. O ensaio sobre a planilha falsa

A planilha falsa tem as 7 abas com título acima do cabeçalho, como é comum em planilha de verdade. Ela traz:

- o mesmo cliente com três nomes;
- uma obra sem código;
- a quantidade fabricada vazia;
- um item com duas linhas de execução;
- uma NF "Atrasada";
- links do Drive em hiperlink;
- e quatro linhas que **têm** de ser ignoradas: unidade inválida, instalada acima do total, pagamento de NF inexistente e valor a descontar acima do valor.

O ensaio confere **17 passos** (antes 9, antes das decisões do 14.1): a leitura, a junção dos clientes, as 4 ignoradas com o motivo certo, os totais, a gravação de **28 registros** (1 cliente, 3 obras, 4 propostas, 4 itens, 2 contratos, 3 itens de contrato, 1 aditivo, 4 execuções, 2 NFs, 2 pagamentos, 2 FDs), a segunda gravação sem duplicar (0 registros), o status das NFs pelo trigger, a cascata no banco, a conciliação (14.3) e o `desfazer` sem sobra. Uma rodada interrompida no meio fica limpa com `verificar-sobras.mjs --apagar`, que agora também acha FD, execução e cliente da importação. Conferido de propósito: gravei sem desfazer e as 15 sobras foram achadas e apagadas.

4b. Dois testes que dependiam do relógio e de dado de terceiros

A primeira rodada do bloco revelou duas falhas que não vinham do código. Os dois seeds foram reaplicados em gc-dev.

- **Dashboard do comercial sem "Documentos em revisão humana".** Em 12/10, o corte da trilha da automação descartou 31 documentos de teste ("Descartado: teste da automação, encerrado no corte"), e o gc-dev ficou sem nenhum em revisão. O teste do 12.3 dependia, por acaso, desses documentos reais. Agora o `seed_conferencia_12_5.sql` cria o próprio, `SEED-DOC-REV`. Se a automação fizer outro corte, vale preservar o que tem `[SEED-TESTE]`.
- **Export de NFs vencidas (9.6) trazendo a SEED-NF-003.** O seed das NFs usa datas relativas ao dia em que foi aplicado (29/09), e o `on conflict do nothing` nunca as atualizava. Em 14/10 a 003, que vencia "em 15 dias", venceu. O `seed_notas_fiscais.sql` agora realinha as datas a hoje a cada aplicação, exceto a cancelada, que é imutável. Pendente: o seed só se realinha quando é reaplicado, então outra data relativa pode vencer no futuro.

5. Validação

Estado: mecanismo validado em 2026-10-19 (a rodada do "Esqueci minha senha", ver `esqueci-minha-senha-status-entrega.md`): estático, unitário (**409 casos**), build (**65 rotas**), runtime (**319/319**), dados (**63/63**), escrita (**520/520**, com o ensaio da importação **9/9**) e navegador (**266/266**).

Rodada final (decisões do 14.1 aplicadas): validado em 2026-10-27, as sete camadas passam.

Rodada final de 2026-10-27 (`caffeinate -ims npm run validar`): passou nas sete camadas.

CAMADA	RESULTADO	DESTE BLOCO
estático	limpo (`tsc --noEmit` + `next lint`)	as decisões do 14.1 compiladas
unitário	**432 casos**	`importacao.test.ts` dentro dos 432
build	**66 rotas**, iguais à baseline	sem rota nova
runtime	**327/327 rotas** + **348 células** da varredura de URL direta	sem rota nova
dados	**65/65 checagens** (1 sem registro em gc-dev); matriz de RLS: **456 células** de tabela e **108** de bucket	sem query nova
escrita	**529/529 passos**, com o ensaio da importação **17/17** dentro dela	ensaio da importação **17/17**, com o importador já com as decisões do 14.1
navegador	**281/281 passos**; screenshots em `/tmp/gc-validacao/shots`	sem tela nova

Bloco fechado em 2026-10-27; a planilha real está fora do alcance (decisão do breno, 27/10).

As rodadas anteriores:

RODADA	ATÉ ONDE FOI	POR QUE PAROU
1	runtime, 318/319 rotas	dashboard do comercial sem documento em revisão (seção 4b, dado de terceiros)
2	escrita, 512/513 passos	export de NFs vencidas trazendo a 003 (seção 4b, data do seed)
3	estático, unitário (**407 casos**), build (**65 rotas**), runtime (**319/319**), dados (**62/62**) passaram; escrita parou	**o DNS da rede local deixou de resolver o endereço do gc-dev** no meio da camada (`Could not resolve host`; o roteador 192.168.0.1 responde SERVFAIL, e o DNS do Google resolve). O gc-dev estava `ACTIVE_HEALTHY`

Rodada 3:

- A queda de rede pegou também a limpeza da escrita, então **ficaram sobras de teste em gc-dev**. Elas saem com `verificar-sobras.mjs --apagar` quando a rede voltar.
- O ensaio da importação (9/9) não chegou a rodar dentro da rodada, porque vem depois da escrita; passou isolado, contra o gc-dev.
- Na camada dados, a checagem "[automação · aditivo] propostas: aditivos de um contrato" voltou com zero linha: **não exercitada**. É da trilha da automação.

Depois da rodada 3, o gc-dev foi pausado por inatividade e reativado em 19/10. As sobras daquela rodada foram apagadas (57).

6. Decisões aplicadas e o que falta, e de quem

- **A planilha real (fora do alcance (decisão do breno, 27/10)):** com ela, seria preciso ajustar o mapa, rodar a simulação e ler as ignoradas e os avisos. O mapa atual é palpite, validado só na planilha falsa.
- **Decidido no 14.1** (por análise, por delegação do Breno em 26/10: "a equipe não vai tomar as decisões"; ver `14.1-status-entrega.md`). **Aplicadas no código em 26/10** (ensaio 17/17; gravação de 28 registros; segunda rodada com 0):**
  1. clientes: "EF Empreendimentos Imobiliários", com equivalência de "Estação Fashion";
  2. contratos: um por obra com proposta aprovada, e aditivos para as outras aprovadas;
  3. execução: fab = ent = máx(fab, inst), med = 0 salvo coluna explícita, com aviso de NF de medição sem medido;
  4. Drive: link na observação na importação; `baixar-drive.mjs` para os 32 arquivos no 14.6, comprovantes primeiro;
  5. corte: dia útil anterior à virada; alvo de virada 13/11/2026 com `--ate 2026-11-12`; reserva 27/11 (corte 26/11).
- **Consequência da decisão 2:** o importador deixa de gravar execução em itens de proposta e passa a gerar um contrato por obra com proposta aprovada, com a execução nos itens do contrato (assim o botão "criar execuções faltantes" do 7.2 enxerga as obras). Mudança **aplicada**.
- **Valor da proposta:** o trigger do 5.6 recalcula o valor da proposta pela soma dos itens. Se a planilha tiver valor diferente da soma dos itens, vale a soma, e a conciliação (14.3) aponta a diferença.
- **Produção:** o script só roda em gc-dev (trava `gc-dev-guard`). gc-prod fora do escopo por decisão do Breno (26/10): o bloco fecha em gc-dev; o restante é registro para quem for operar prod. O ensaio do go-live (14.4) roda em gc-dev.
- **Validação final:** validado em 2026-10-27 (seção 5). Fora do alcance: a planilha real.